

This is a good **Day 7** progression because it moves from JavaScript into the typed language you'll commonly encounter in Playwright automation.

## Day 7 — TypeScript Fundamentals for Test Automation

### 1. JavaScript vs TypeScript

| JavaScript                             | TypeScript                                |
|----------------------------------------|-------------------------------------------|
| Dynamically typed                      | Statically typed                          |
| Errors often appear at runtime         | Many errors caught before execution       |
| <code>.js</code> files                 | <code>.ts</code> files                    |
| No explicit type declarations required | Types can be explicitly declared          |
| Browser can execute it directly        | Usually compiled/transpiled to JavaScript |
| Common in web development              | Very common for test automation           |

Example:

#### JavaScript

```
// JavaScript
let age = 25;
age = "twenty five";
```

JavaScript allows this.

#### TypeScript

```
// TypeScript
let age: number = 25;
age = "twenty five";
```

TypeScript reports an error because `age` was declared as a `number`.

### 2. Why Playwright Commonly Uses TypeScript

Playwright supports both JavaScript and TypeScript, but TypeScript is particularly useful for automation because:

- Test data can have defined types.
- Function parameters can be validated.

- Return values can have known types.
- IDEs provide better autocomplete.
- Many mistakes are detected before running tests.
- Interfaces make complex test data easier to understand.
- Large automation frameworks become easier to maintain.

For example:

#### TypeScript

```
interface Customer {  
  name: string;  
  age: number;  
  email: string;  
}  
  
const customer: Customer = {  
  name: "Ravi",  
  age: 35,  
  email: "ravi@test.com"  
};
```

The IDE and compiler can immediately identify missing or incorrectly typed fields.

## 3. TypeScript Project Basics

A simple Playwright TypeScript project may look like:

```
playwright-project/  
|  
├─ tests/  
│   └─ login.spec.ts  
|  
├─ pages/  
│   └─ LoginPage.ts  
|  
├─ test-data/  
│   └─ users.ts  
|  
├─ playwright.config.ts  
├─ package.json  
└─ tsconfig.json
```

The important point for today is:

**.ts = TypeScript source file**

Example:

TypeScript

```
const browserName: string = "Chrome";
```

---

## 4. Variables with Types

### String

TypeScript

```
let firstName: string = "Ravi";
```

### Number

TypeScript

```
let age: number = 35;
```

### Boolean

TypeScript

```
let isActive: boolean = true;
```

### Multiple variables

TypeScript

```
let name: string = "Ravi";  
let age: number = 35;  
let isEmployee: boolean = true;
```

TypeScript can also infer the type:

TypeScript

```
let name = "Ravi";  
let age = 35;  
let isEmployee = true;
```

TypeScript understands:

```
name      → string
age       → number
isEmployee → boolean
```

---

## 5. Arrays

### String array

TypeScript

```
let browsers: string[] = [
  "Chrome",
  "Firefox",
  "Edge"
];
```

### Number array

TypeScript

```
let ages: number[] = [25, 30, 35, 40];
```

Another syntax is:

TypeScript

```
let browsers: Array<string> = [
  "Chrome",
  "Firefox",
  "Edge"
];
```

For automation, arrays are very common:

TypeScript

```
const usernames: string[] = [
  "admin",
  "tester",
  "manager"
];
```

---

## 6. Objects

You can define an object directly:

TypeScript

```
const user = {  
  name: "Ravi",  
  age: 35,  
  active: true  
};
```

TypeScript infers the types.

You can also explicitly define them:

TypeScript

```
const user: {  
  name: string;  
  age: number;  
  active: boolean;  
} = {  
  name: "Ravi",  
  age: 35,  
  active: true  
};
```

This becomes cumbersome for larger objects.

That's where **interfaces** become useful.

---

## 7. Basic Interfaces

TypeScript

```
interface User {  
  name: string;  
  age: number;  
  email: string;  
}
```

Now create an object:

TypeScript

```
const user: User = {
```

```
name: "Ravi",
age: 35,
email: "ravi@test.com"
};
```

If you accidentally write:

TypeScript

```
const user: User = {
  name: "Ravi",
  age: "thirty five",
  email: "ravi@test.com"
};
```

TypeScript reports an error because:

age must be a number

---

## 8. Functions

JavaScript:

JavaScript

```
function add(a, b) {
  return a + b;
}
```

TypeScript:

TypeScript

```
function add(a: number, b: number): number {
  return a + b;
}
```

The syntax is:

```
function functionName(parameter: type): returnType
```

Example:

TypeScript

```
function greet(name: string): string {
```

```
    return "Hello " + name;
}
```

## 9. Function Parameters and Return Types

Consider an insurance example:

TypeScript

```
function calculatePremium(
    age: number,
    coverage: number
): number {

    return coverage * 0.05 + age * 10;
}
```

Calling it:

TypeScript

```
const premium = calculatePremium(35, 500000);

console.log(premium);
```

The compiler knows:

```
age        → number
coverage   → number
premium    → number
```

If you write:

TypeScript

```
calculatePremium("35", 500000);
```

TypeScript reports an error.

## 10. Optional Values

Sometimes a value isn't mandatory.

Use `?`.

### TypeScript

```
interface Customer {  
  name: string;  
  age: number;  
  email?: string;  
}
```

Now both are valid:

### TypeScript

```
const customer1: Customer = {  
  name: "Ravi",  
  age: 35,  
  email: "ravi@test.com"  
};
```

and:

### TypeScript

```
const customer2: Customer = {  
  name: "Kumar",  
  age: 40  
};
```

`email` is optional.

---

## 11. Type Safety

This is one of the most important concepts for testers.

Suppose:

### TypeScript

```
let age: number = 35;
```

This is valid:

### TypeScript

```
age = 40;
```

This is invalid:



TypeScript

```
age = "forty";
```

TypeScript catches the problem before the test is executed.

That is especially useful in automation because a mistake in test data can otherwise result in confusing runtime failures.

---

## 12. Understanding Compiler Errors

Consider:

TypeScript

```
let age: number = 25;  
  
age = "25";
```

You might see an error similar to:

```
Type 'string' is not assignable to type 'number'.
```

Read it from right to left:

```
You have a string  
      ↓  
but TypeScript expects a number
```

Another example:

TypeScript

```
function login(username: string, password: string) {  
    // ...  
}  
  
login("admin", 12345);
```

Error:

```
Argument of type 'number' is not assignable  
to parameter of type 'string'.
```

The tester should identify:

Expected → string

Actual → number

## 13. Reading TypeScript Automation Code

Consider this Playwright-style example:

TypeScript

```
import { test, expect } from "@playwright/test";

interface LoginData {
  username: string;
  password: string;
}

const loginData: LoginData = {
  username: "admin",
  password: "admin123"
};

test("Login Test", async ({ page }) => {

  await page.goto("https://example.com");

  await page.getByLabel("Username")
    .fill(loginData.username);

  await page.getByLabel("Password")
    .fill(loginData.password);

  await page.getByRole("button", {
    name: "Login"
  }).click();

});
```

A tester should be able to identify:

### Interface

TypeScript

```
interface LoginData
```

Defines the structure of the test data.

## Properties

TypeScript

```
username: string;  
password: string;
```

Both properties must contain strings.

## Test data

TypeScript

```
const loginData: LoginData
```

The object must follow the `LoginData` structure.

## Async function

TypeScript

```
async ({ page }) => {
```

The test performs asynchronous browser operations.

## Playwright locator

TypeScript

```
page.getByLabel("Username")
```

Finds the username field.

## Test data usage

TypeScript

```
loginData.username
```

Retrieves the username from the typed object.

---

## 14. Hands-on Exercise 1 — Convert JavaScript to TypeScript

## JavaScript

### JavaScript

```
let name = "Ravi";
let age = 35;
let active = true;

function greet(name) {
  return "Hello " + name;
}

console.log(greet(name));
```

## Your task

Convert this into TypeScript by adding:

- Variable types
- Parameter type
- Return type

Expected structure:

### TypeScript

```
let name: string = "Ravi";
let age: number = 35;
let active: boolean = true;

function greet(name: string): string {
  return "Hello " + name;
}

console.log(greet(name));
```

---

## 15. Hands-on Exercise 2 — Typed Test Data

Create an interface:

### TypeScript

```
interface InsuranceApplicant {
  name: string;
  age: number;
  email: string;
  smoker: boolean;
```

Create test data:

TypeScript

```
const applicant: InsuranceApplicant = {  
  name: "Ravi",  
  age: 35,  
  email: "ravi@test.com",  
  smoker: false  
};
```

Now deliberately introduce errors:

TypeScript

```
const applicant: InsuranceApplicant = {  
  name: "Ravi",  
  age: "35",  
  email: "ravi@test.com",  
  smoker: "No"  
};
```

## Identify the errors

You should find:

```
age    → should be number  
smoker → should be boolean
```

---

## 16. Hands-on Exercise 3 — Insurance Premium Function

Create:

TypeScript

```
function calculatePremium(  
  age: number,  
  coverage: number  
): number {  
  
  return coverage * 0.05 + age * 10;  
}
```

Test it:

TypeScript

```
const premium = calculatePremium(35, 500000);

console.log(premium);
```

Then deliberately make errors:

TypeScript

```
calculatePremium("35", "500000");
```

Use the compiler error messages to determine what is wrong.

---

## 17. Hands-on Exercise 4 — Optional Test Data

Create:

TypeScript

```
interface Applicant {
  name: string;
  age: number;
  email?: string;
}
```

Test:

TypeScript

```
const applicant1: Applicant = {
  name: "Ravi",
  age: 35
};
```

Then:

TypeScript

```
const applicant2: Applicant = {
  name: "Kumar",
  age: 40,
  email: "kumar@test.com"
};
```

Both should compile successfully.

## 18. Hands-on Exercise 5 — Find the Type Errors

Identify all errors:

TypeScript

```
let applicantName: string = "Ravi";

let applicantAge: number = "35";

let isSmoker: boolean = "false";

let coverage: number = 500000;

function calculate(age: number): string {
    return age + 10;
}
```

### Questions

1. Why is `applicantAge` invalid?
2. Why is `isSmoker` invalid?
3. Why is the return type of `calculate()` incorrect?
4. Correct the code.

## Day 7 Learning Outcome

By the end of this session, a tester should be comfortable reading code such as:

TypeScript

```
interface TestUser {
    username: string;
    password: string;
    age?: number;
}

function createUser(
    username: string,
    password: string
): TestUser {

    return {
        username,
        password
    };
}
```

```
}  
  
const user: TestUser = createUser(  
  "admin",  
  "admin123"  
);
```

The key skill is **not becoming a TypeScript developer in one day**. The goal is to be able to **read, write, debug, and modify TypeScript automation code confidently**, which provides the foundation for the Playwright portions of the training.



 Oracle

Ad

Cloud Native on OCI  
OKE and OCI Functions. Start free.